

Verwendung von Arrays

	eindimensional	zweidimensional
Deklaration:	`int[] zahlen;`	`Color[][] pixel;`
Initialisierung:	`zahlen = new int[20];`	`pixel = new Color[100][50];`
Zugriff auf ein Arrayelement:	`zahlen[5] = 10;` `zahlen[i] = 12;`	`pixel[4][7] = Color.BLACK;` `pixel[x][y] = Color.BLUE;`
Direkte Initialisierung:	`char[] buchstaben = {'i','n','f'};`	`double[][] faktoren = {{ 4.0, -1.2, 2.3}, { -2.1, 2.1, 0.0}};`

Achtung: Bei zweidimensionalen, direkt initialisierten Arrays gibt die erste Position die Zeile und die zweite die Spalte an: `faktoren[0][2]` hat im Beispiel oben den Wert 2,3.

Auftrag:

1. *Gib die notwendigen Java-Befehle an, um ...*

a) *ein Array zu deklarieren und zu initialisieren, um 35 Kommazahlen zu speichern. Welche Position hat das erste bzw. das letzte Element des Arrays?*

b) *ein zweidimensionales Array für ein Bild der Größe 800x350 Pixel zu deklarieren und zu initialisieren.*

c) *den 4. Wert des eindimensionalen Arrays auf 45,2 zu setzen und den 3. Wert auf 13,2.*

d) *den 5. Wert des eindimensionalen Arrays auf die Summe des 3. und des 4. Wertes zu setzen.*

e) *das Pixel an der Stelle x = 17 und y = 34 auf Rot zu setzen.*

f) *dem Pixel an der Stelle x = 123 und y = 12 die Farbe des Pixels an der Stelle x = 76 und y = 99 zuzuweisen.*

Folgender Java-Code (in Processing) kopiert die Pixel eines Bildes sowohl in ein zweidimensionales Array als auch in ein neues Bild.

```
int breite = originalbild.width;
int hoehe = originalbild.height;

pixelNeu = new color[breite][hoehe];
PImage neuesBild = createImage(breite, hoehe, RGB);

for (int x = 0; x < breite; x++) {
  for (int y = 0; y < hoehe; y++) {
    pixelNeu[x][y]= pixelAlt[breite-1-x][y];
    neuesBild.set(x, y, pixelNeu[x][y]);
  }
}
```

Die verschachtelte Zählschleife ermöglicht es, jedes Pixel des neuen Bildes zu definieren.

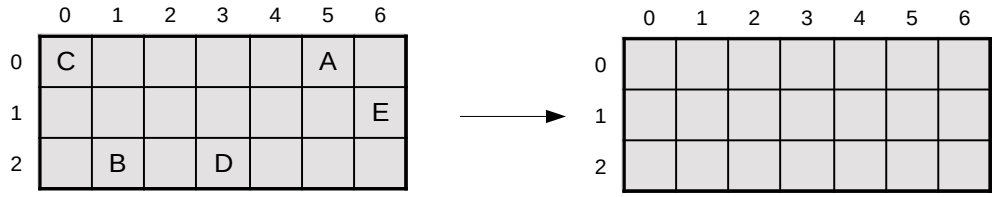
Auftrag:

2. *Gib an, in welcher Reihenfolge sie gesetzt werden, falls das Bild 7x3 Pixel groß ist:*

Durchgang	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
x																					
y																					

3. *Gib an, welches Pixel des Originalbildes an die Position (5|2) wandert. Woher kommt das Pixel mit der Position (0|0)? Wohin geht das Pixel, das ursprünglich an der Position (3|1) im Originalbild war?*

4. *Zeichne ein, an welcher Stelle die Pixel A- E im neuen Bild zu finden sind.*



5. *Beschreibe mit 3-4 Worten, wie das neue Bild aus dem alten hervorgeht.*

Programmierauftrag

6. *Öffne das Projekt "p01bildbearbeitung" in Processing. Starte das Projekt. Beschreibe, was in der setup()-Methode passiert.*

7. *Spiegle das geladene Bild horizontal. Rufe dazu in der setup()-Methode die Methode spiegleHorizontal auf: bildNeu = spiegelHorizontal(bildAlt). Nach dem Spiegeln musst du das neue Bild mit image() wieder anzeigen lassen. Achte darauf, dass es neben dem alten Bild angezeigt wird.*

8. *Speichere das neue Bild mit save(„data/spiegelsee.jpg“). Überprüfe im Projektordner, ob das Speichern geklappt hat.*

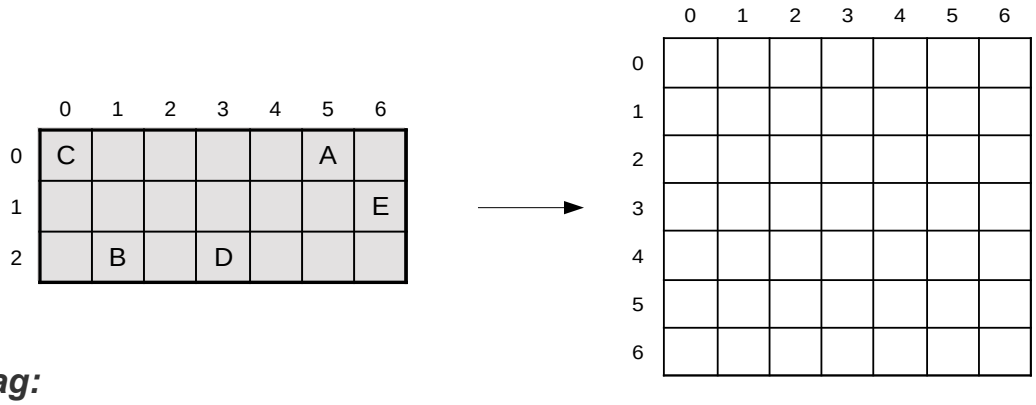
9. *Teste dein Programm auch mit einem anderen Bild aus dem data-Ordner.*

10. *Implementiere die Methode* `PImage spiegleVertikal(PImage originalbild)` *analog zur spiegleHorizontal-Methode.*

    Teste deine neue Methode.

Drehung des Bildes um 90°

Ein Bild soll um 90° nach links gedreht werden.



Auftrag:

11. *Markiere im Raster den Bereich, den das gedrehte Bild abdeckt. Wie kann die Breite und die Höhe des neuen Bildes bestimmt werden?*
12. *Zeichne ein, an welcher Stelle die Pixel A-E im neuen Bild zu finden sind.*
13. *Bestimme die Koordinaten der Punkte A-E im neuen Bild und im Originalbild.*

	A	B	C	D	E
x_{neu}	0				
y_{neu}	1				
$x_{original}$	5				
$y_{original}$	0				

14. *Ermittele Formeln, mit denen die Koordinaten des ursprünglichen Pixel bestimmt werden können:*

    $x_{original}$ =

    $y_{original}$ =

    In Java: `pixelNeu[x][y] = pixelAlt[ ][ ];`

Programmierauftrag:

15. *Implementiere die Methode* `PImage dreheLinks(PImage originalbild).`
16. *Stelle die gleichen Überlegungen (Aufgaben 11-14) für eine Drehung um 90° nach rechts an. Implementiere die Methode* `PImage dreheRechts(PImage originalbild).`
17. *Überlege, wie du mit den bestehenden Methoden eine Drehung um 180° realisieren könntest. Teste dies in Processing durch geeignete Methodenaufrufe. Implementiere eine Methode* `PImage drehe180(PImage originalbild)` *unter Verwendung der bestehenden Methoden.*
    Hinweis: das Zwischenergebnis darf auch als Bild angezeigt werden.